TCPThreadPoolServer Project Explanation

Project Overview

This project is a C++ systems-programming example that combines a TCP socket server with a custom thread pool. It demonstrates socket creation, binding, listening, accepting connections, handling client requests, and concurrent task execution using threads.

Main Components

1. TCP Server: listens on port 8080, accepts incoming client connections, receives a message, sends a response, and closes the client socket.
2. Thread Pool: maintains multiple worker threads and a shared task queue. It demonstrates synchronization with std::mutex and std::condition_variable.

Architecture

The project is structured into two separate but related systems. The server is built in src/Tcpserver.cpp and the thread pool logic in src/ThreadPool.cpp. The main entry point starts the pool and then the server in src/main.cpp.

Key Files

- CMakeLists.txt: defines the build target and C++17 requirement.
- include/TCPserver.h: declares the TCP server interface.
- include/ThreadPool.h: declares the thread pool API.
- src/Tcpserver.cpp: contains the socket logic and client handling.
- src/ThreadPool.cpp: contains worker execution and queue logic.
- src/main.cpp: application entry point.

Technology Stack

- C++17
- POSIX sockets
- std::thread
- std::mutex
- std::condition_variable
- std::queue
- std::function
- CMake
- Ubuntu/WSL or Docker for Linux-compatible execution

Requirements to Run

The project is intended for a Linux or WSL environment because it uses Unix socket APIs such as socket(), bind(), listen(), accept(), recv(), close(), and pthread-based threading. The machine must have a C++ compiler, CMake, and access to port 8080.

Build Instructions

From the project root:
mkdir -p build
cd build
cmake ..
cmake --build .
./tcp_server

Docker Usage

This repository includes a Dockerfile. Build and run with:
docker build -t tcp-threadpool-server .
docker run -p 8080:8080 tcp-threadpool-server

How It Works

When the server starts, it creates a socket, sets socket options, binds to port 8080, and begins listening. For each client connection, it accepts the client, receives the message, responds with HELLO WORLD, and closes the connection. The thread pool demonstrates how tasks are scheduled across worker threads while the server itself runs independently.

Conclusion

This project is a practical demonstration of systems programming concepts: TCP communication, concurrency, synchronization, and task scheduling. It is ideal for learning how low-level network services and multithreaded task execution are implemented in C++.